



SELECTED WORK · 04

LR Car Services

The **driver's app** for a dispatcher-run car service — built and shipped by me.

A dispatcher assigns a trip; the driver's phone takes it from there. Multi-stop routes, waiting time counted separately from driving time, and a location feed that keeps reporting even when the app has been killed.

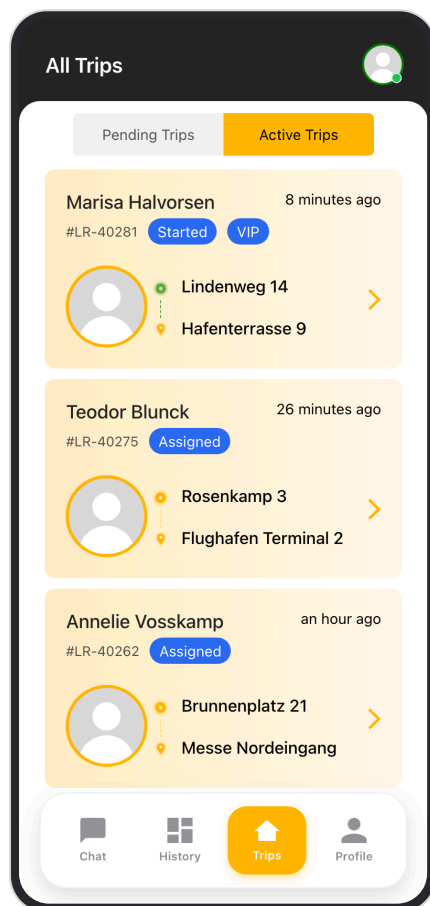
My role Everything: the React Native app, the cloud functions, the data model and the releases.

Mobile, Android + iOS

Real-time dispatch

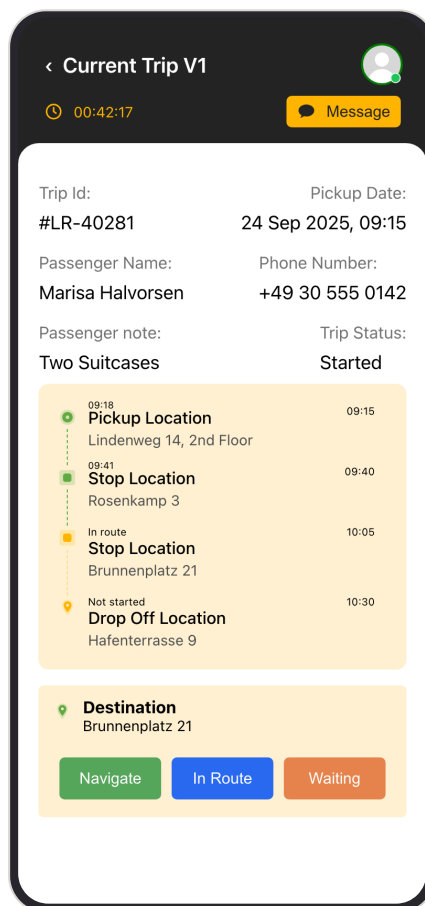
Background GPS

Push & chat



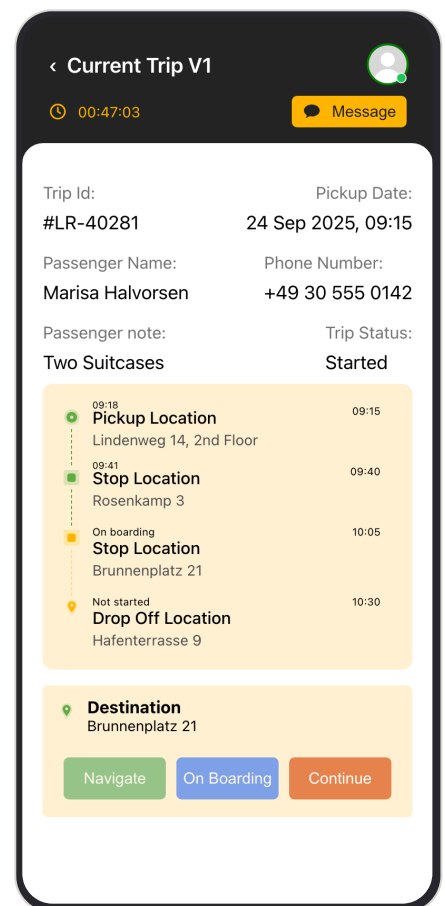
The day's work

Assigned and active trips, four tabs



A trip in progress

Pickup, stops and drop-off, each with its own state



Waiting at a stop

The clock keeps running, on a separate meter

Screens drawn from the app's own source — every label, state and button is one that ships

20

screens in the driver app

5

separate cloud services behind it

60s

without a ping and a driver shows offline

v8

shipped to real drivers on both stores

React Native

React 19

TypeScript

Firebase Auth

Firestore

Cloud Functions

FCM push

Background GPS

Google Cloud



THE TRIP ITSELF

Five states, one direction

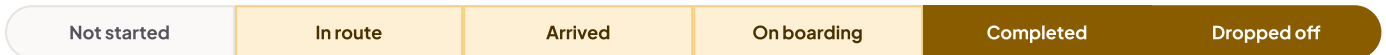
The dispatcher opens it, the driver moves it forward, nobody skips a step



AND INSIDE IT

Every stop has its own six

Pickup, stop, drop-off — each tracked, each timed



< Current Trip V1

Message

Trip Id: #LR-40281
Pickup Date: 24 Sep 2025, 09:15
Passenger Name: Marisa Halvorsen
Phone Number: +49 30 555 0142
Passenger note: Two Suitcases
Trip Status: Completed

09:18 Pickup Location
Lindenweg 14, 2nd Floor

09:41 Stop Location
Rosenkamp 3

10:12 Stop Location
Brunnenplatz 21

10:37 Drop Off Location
Hafenterrasse 9

Waiting Time: 00:14:22
Total Time: 01:22:05

Back To Trips

WHY WAITING IS COUNTED ON ITS OWN

A chauffeur job is half driving and half waiting — outside a meeting, at an airport, at a restaurant. The driver taps **Waiting**, and taps **Continue** when they move again. Each pause is stored, then added up.



Two figures, not one. The office can bill the wait, and a driver who sat for forty minutes can prove it.

Partial drop-offs

Passengers can leave at different stops — the trip carries on without them

Old trips keep the old app

Two generations of the trip screen live side by side, chosen per trip

A line to the office

Driver and dispatcher chat inside the trip; both sides get a notification

THE HARD PART OF A DRIVER APP

A phone in a pocket, all day

Killed apps, dead batteries, tunnels — none of it can lose the trip

THE PHONE

Driver app

Keeps reporting after the app is closed, and starts itself on reboot

Batched, not chatty

Points are queued and sent together, signed

THE SERVICES

Location service

Takes the batch, writes the driver's latest position

Every minute, a check

No ping for 60 seconds and the driver is marked offline

Push, chat, assignment

A new trip or a new message opens the right screen

WHO SEES IT

The dispatcher

Sees who is online, who is where, and who is late

Only a dispatcher

Reassigning a driver is refused for anyone else

- ✓ **Reassignment is a privilege.** The server checks the caller is a dispatcher and refuses everyone else — the app being polite is not enough.
- ✓ **Drivers are approved, not just registered.** Pending, approved or blocked, decided by the office.
- ✓ **Addresses are not hard-coded.** The app reads where its services live at start-up, so an endpoint can move without a store release.
- ✓ **Updates can be made compulsory.** A driver on a broken old build is a lost trip, so Android can force the update.
- ✓ **Trips already running never change shape.** A new trip screen shipped beside the old one instead of replacing it.

DECISIONS I'D DEFEND

Waiting time	its own number, from day one
A trip	a chain of stops, not A to B
Location	batched and signed, not streamed
Offline	decided by the server's clock
Old versions	kept working, not forced out

Trips the work itself

Drivers who may drive

Driver status online, permissions

Locations where they are

Chats office and road

Settings what the app reads